

Reproducible pandas pipelines

We have downloaded the evidence.
How do we transform it into reliable results
without silently making it wrong?



Begin with the exact Lecture 4 evidence pack



No manual copying. No new download. No silent substitution.

Notebook cells 2–3 • a changed persisted input stops execution before analysis.

Write the transformation contract first

Field	Contract for the real case
Grain	Brent: reported date → month; PBS: indicator × month
Keys	Unique raw date; unique oil month; unique PBS composite key
Units	USD / barrel and CPI index remain separate columns
Missingness	Missing quote ≠ malformed token ≠ invalid value; never zero-fill
Deliverables	Joined table, clean lab, quarantine, issue log and run record

Parse deliberately and keep the raw tokens

```
raw = pd.read_csv(path, dtype='string',  
                  keep_default_na=False)  
missing = raw['DCOILBRENTU'].eq('')  
value = pd.to_numeric(  
    raw['DCOILBRENTU'].mask(missing),  
    errors='coerce')  
malformed = ~missing & value.isna()
```

Three decisions

Raw token retained.

Blank is missing.

Nonblank parse failure is a different event.

The source uses reported observation dates, not a complete daily calendar.

Missing, malformed and invalid are different

Missing

No reported quote
Blank token → NA

Malformed

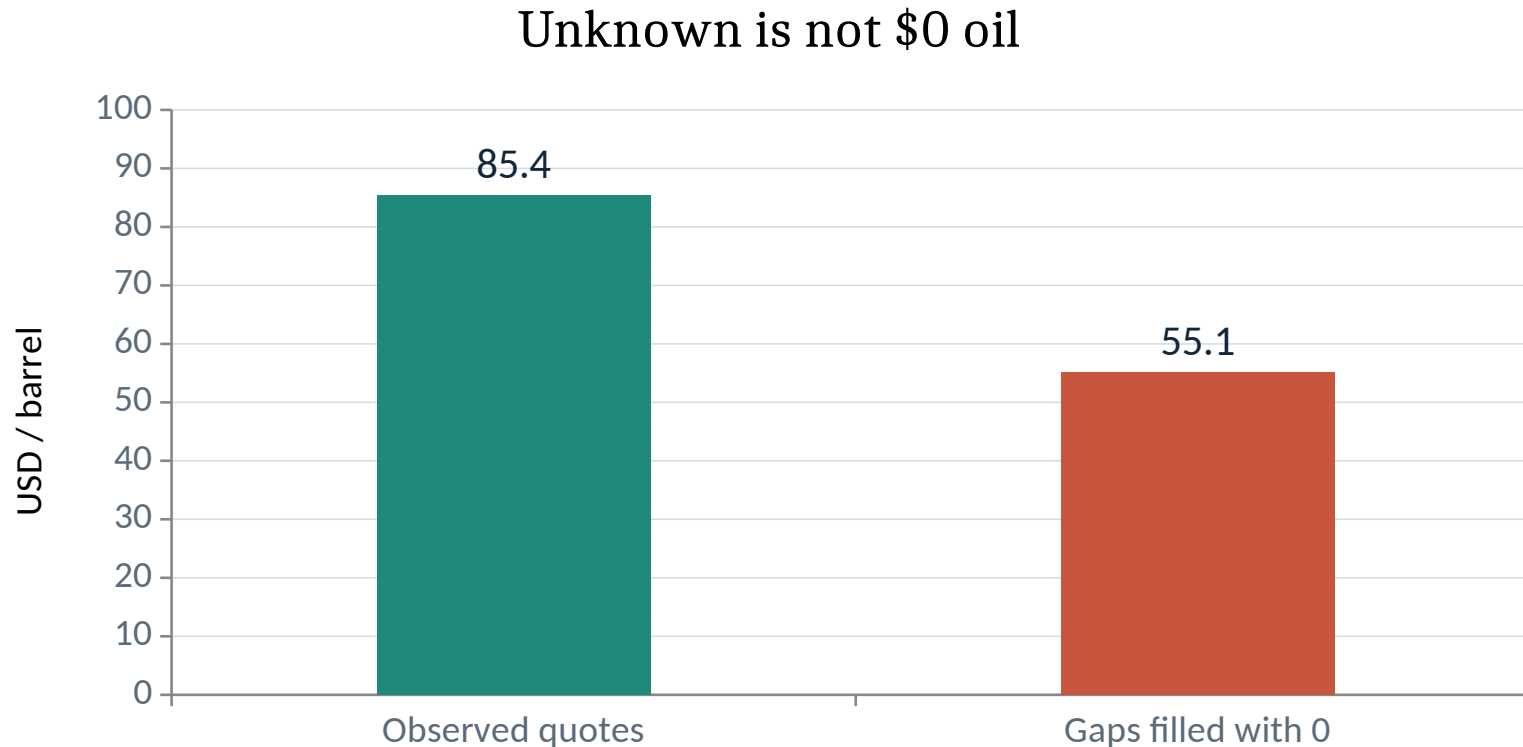
Cannot parse as numeric
“1020” → parse failure

Invalid

Parsed but violates policy
-5 → invalid in this lab

One `isna()` result cannot explain why the value became missing. Synthetic zero remains observed.

The denominator can silently change the answer



State the denominator

20 observed quotes

21 source rows

31 calendar days

Weekends / closures are not zero prices.

Executed real-data example • reindexing is not permission to impute zeros.

Audit real keys before injecting any defects

Date (defect copy)	USD / barrel
2026-09-29	113.96
2026-09-29	999.0

Defect copy only: same date, two values → quarantine both; the raw evidence remains unchanged.

Controlled failure lab: raw records 1–7

Row	Category text	Date text	Value text
1	food	01/01/2024	100
2	food	01/02/2024	110
3	FOOD	01/02/2024	110
4	FOOD	01/03/2024	121
5	transport	01/01/2024	200
6	TRANSPORT	01/02/2024	
7	TRANSPORT	01/03/2024	-5

Synthetic fixture • row 3 is a confirmed copy; blank and –5 need different policies.

Controlled failure lab: raw records 8–15

Row	Category text	Date text	Value text
8	housing	01/01/2024	1,000
9	HOUSING	01/02/2024	not reported
10	HOUSING	01/03/2024	1020
11	FOOD	31/02/2024	130
12	HOUSING	01/01/2024	1,050
13	MISC	01/01/2024	0
14	MISC	01/02/2024	5
15	MISC	01/03/2024	10

Synthetic fixture • 1020 is not 1020; 31/02/2024 is invalid; zero is an observed value.

Normalize identifiers, not evidence away

Row	Raw token	Typed value	Reason
6		<NA>	source_missing
7	-5	<NA>	invalid_negative
9	not reported	<NA>	source_missing
10	1020	<NA>	parse_failure

Actual notebook display · parsed NA values retain different reason codes.

Confirmed copies and key conflicts need policies

Confirmed repeated record

Rows 2 and 3:
FOOD, February, 110

Source note confirms row 3 is an accidental copy. Remove and log.

Unresolved analytical key

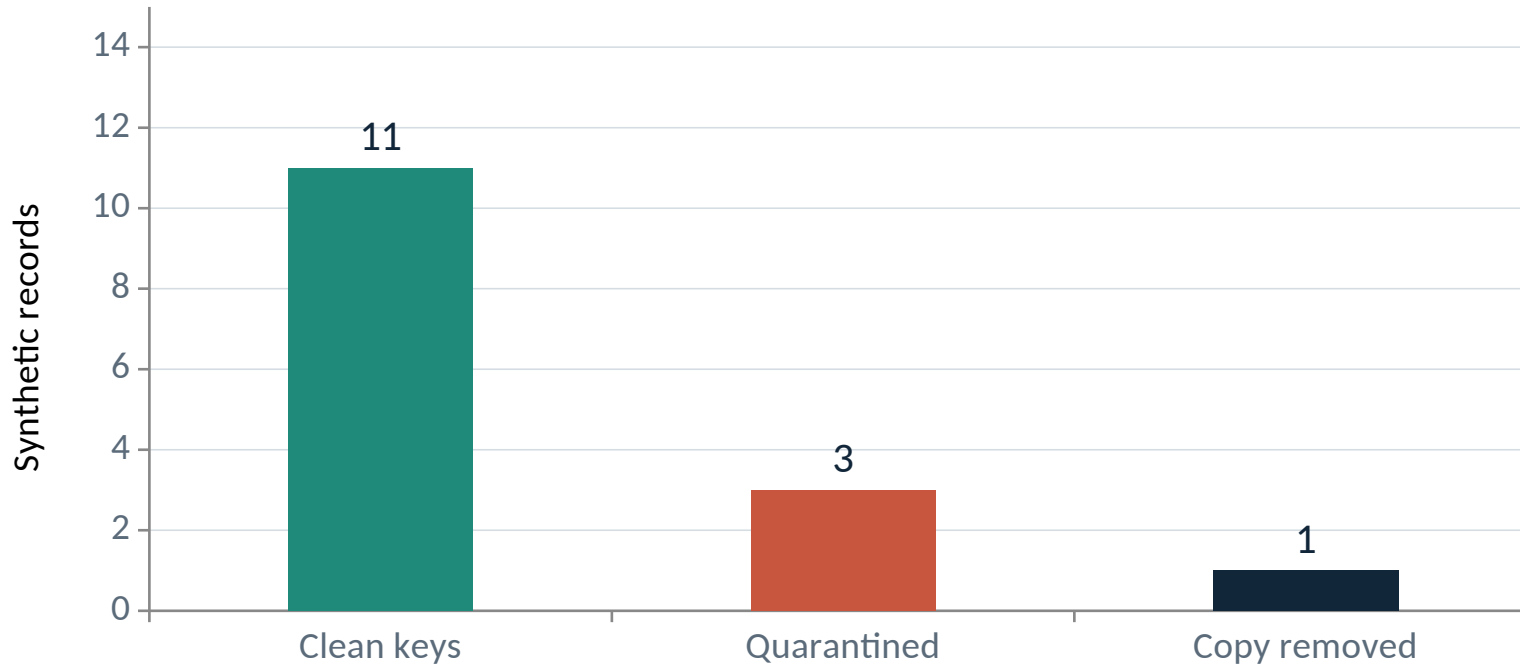
Rows 8 and 12:
HOUSING, January, 1000 / 1050

Quarantine both. Never average or keep first.

Invalid date row 11 is also quarantined; unknown values remain as missing-valued clean records.

Account for every input record

$$15 = 11 + 3 + 1$$



Clean $\neq$ all observed

11 clean keys
7 observed values
4 missing values

8 issue events explain the decisions.

Independent check: staged result == prepare_prices result; input unmutated.

The issue log is part of the result

Row	Issue	Action
3	confirmed_copy	remove
6	source_missing	retain_missing
7	invalid_negative	retain_missing
8	unresolved_key	quarantine
9	source_missing	retain_missing
10	parse_failure	retain_missing
11	invalid_date	quarantine
12	unresolved_key	quarantine

Actual notebook issue log • each saved event also includes a reason.

concat stacks; merge relates

concat: compatible batches

Same columns, units and meaning.
More observations, same grain.

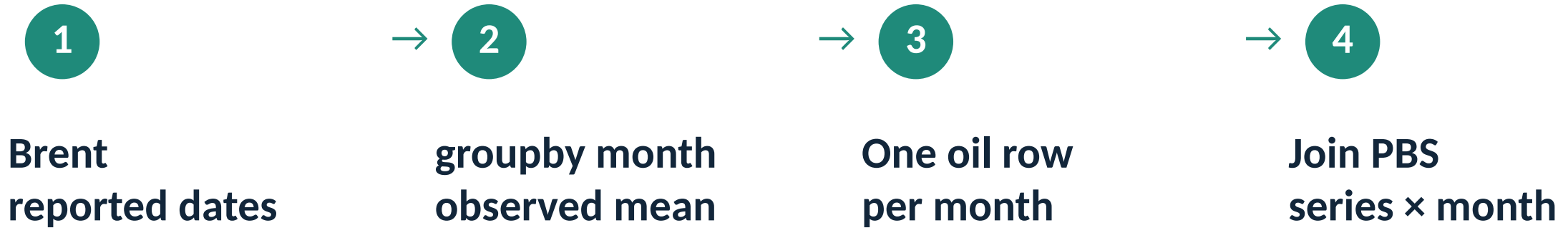
Check schema and key overlap.
A reset index does not deduplicate.

merge: a key relationship

Related tables, explicit join keys.
Potentially different grains.

Check cardinality, coverage and primary-row preservation.

Daily oil must become monthly before the join



Aggregation changes grain. Keep observed quote counts with the mean.

An aggregation should carry its coverage

Month	Mean USD / barrel	Observed	Source rows	Cal. days
2026-06	85.4	22	22	30
2026-07	83.76	23	23	31
2026-08	91.08	20	21	31
2026-09	114.08	21	21	30

count = observed quotes; size = source rows; calendar days $\neq$ either • September is partial.

Validate cardinality and expose match coverage

```
joined = recent.merge(brent_monthly,  
    on='period', how='left',  
    validate='many_to_one', indicator=True)  
  
assert len(joined) == len(recent)  
assert joined['_merge'].eq('both').all()
```

Predict first

3 series × 12 months
36 retained rows
0 unmatched

Keys, count and coverage are
separate checks.

The executed CPI–Brent join

Month	CPI basket	CPI index	Oil USD / barrel	Quotes
2026-08	Food + non-alcoholic drinks	313.99	91.08	20
2026-08	Transport	382.06	91.08	20
2026-08	General	300.50	91.08	20

Same month, separate units • CPI base 2015–16 = 100; Brent USD / barrel • alignment, not causality

One duplicated month multiplies observations

Unchecked join

Duplicate one right-side oil month.

36 primary rows become 39 rows.

The operation runs, but observation weights change.

Validated join

`validate="many_to_one"`

MergeError: right keys not unique.

Fail visibly before a plausible summary hides the defect.

Predict the three extra rows before executing the labelled defect copy.

Valid cardinality does not guarantee coverage

Cardinality passes

At most one oil record per month.

No row multiplication.

The relationship is structurally valid.

Coverage can fail

Restricted oil table: 2026 onward.

24 rows matched; 12 left_only.

Retain and report, reconcile, or explicitly fail.

indicator=True reveals unmatched rows. An inner join would silently change the cohort.

An index level becomes a rate by comparison

```
ordered = pbs.sort_values(['indicator', 'month'])
ordered['yoy_pct'] = (ordered
    .groupby('indicator')['index_2015_16_100']
    .pct_change(12, fill_method=None) * 100)

# YoY = 100 × (It / I(t-12) - 1)
```

Index ≠ inflation

Denominator: same month one year earlier.

First 12 months: no YoY.

Monthly gaps require a calendar audit.

Aggregation and transform keep different grains

agg: group-level output

Daily Brent → one monthly mean.

Return the mean and observed count.

Output grain changes.

transform: row-level output

Annual mean broadcast to daily rows.

Daily deviation = quote – annual mean.

Original grain is retained.

Mean of means and pooled mean answer differently

Equal month weights

2024 mean of monthly means:
80.53 USD / barrel

Each month counts equally.

Board check: $(70 + 100) / 2 = 85$

Equal quote weights

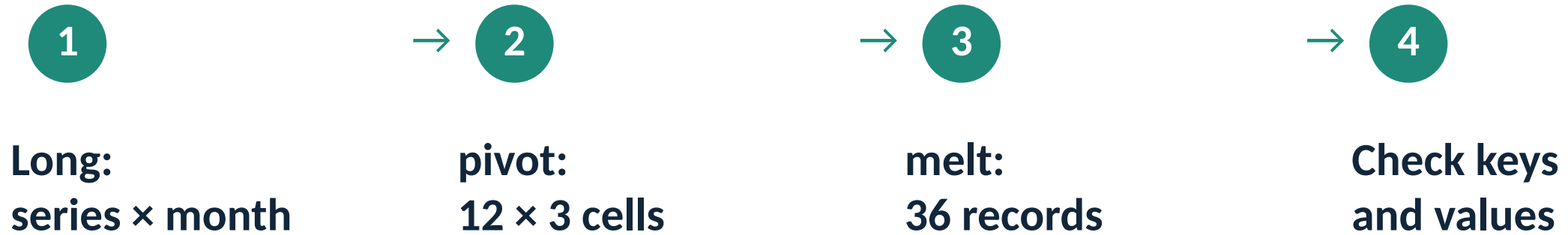
2024 pooled observed mean:
80.52 USD / barrel

Each observed quote counts equally.

Board check: $(3 \times 70 + 2 \times 100) / 5 = 82$

Choose weights according to the question, not according to whichever result looks convenient.

pivot and melt change layout, not meaning



Unique cell keys are the contract; a reshape is not a reconciliation policy.

An ambiguous pivot cell is not an average

pivot: expose the ambiguity

One cell, two candidate values:
290.74 and 999.00

ValueError: duplicate cell keys.

Reconcile the evidence first.

pivot_table: implicit aggregation

Default mean: 644.87

A tidy-looking number that silently changes the evidence.

Use only a justified aggregation policy.

Package a policy, then independently validate it

```
oil_monthly = (raw.pipe(tidy_brent)
               .pipe(monthly_oil))

assert_frame_equal(oil_monthly, staged)
final = recent.merge(oil_monthly,
                    on='period', how='left',
                    validate='many_to_one', indicator=True)
```

Independent check

Same raw inputs.
Same declared policies.
Staged == packaged result.

Re-check join preservation.

Small functions state contracts; validate against the staged computation, not just “it ran”.

Reproducibility needs more than a CSV

Artifact	What another person can inspect
lab_clean_prices.csv	Unique keys; observed and missing values with reasons
lab_quarantine.csv + lab_issues.csv	Rejected candidates and explicit decisions
cpi_brent_monthly.csv	36-row real join with declared monthly grain
run_record.json	Versions, persisted input path and accounting
Slide asset manifest + build sources	Notebook cells, hashes and reproducible slide assets

Recompute the iPhone benchmark

Launch	Model	USD	PKR / USD	Pre-tax PKR
2019	iPhone 11 Pro Max	1099	156.08	171531.70
2021	iPhone 13 Pro Max	1099	169.08	185820.02
2023	iPhone 15 Pro Max	1199	291.76	349823.72
2026	iPhone 18 Pro Max	1299	277.25	360150.61

Recomputed benchmark: +110.0% since 2019 • pre-tax arithmetic, not Pakistan retail prices

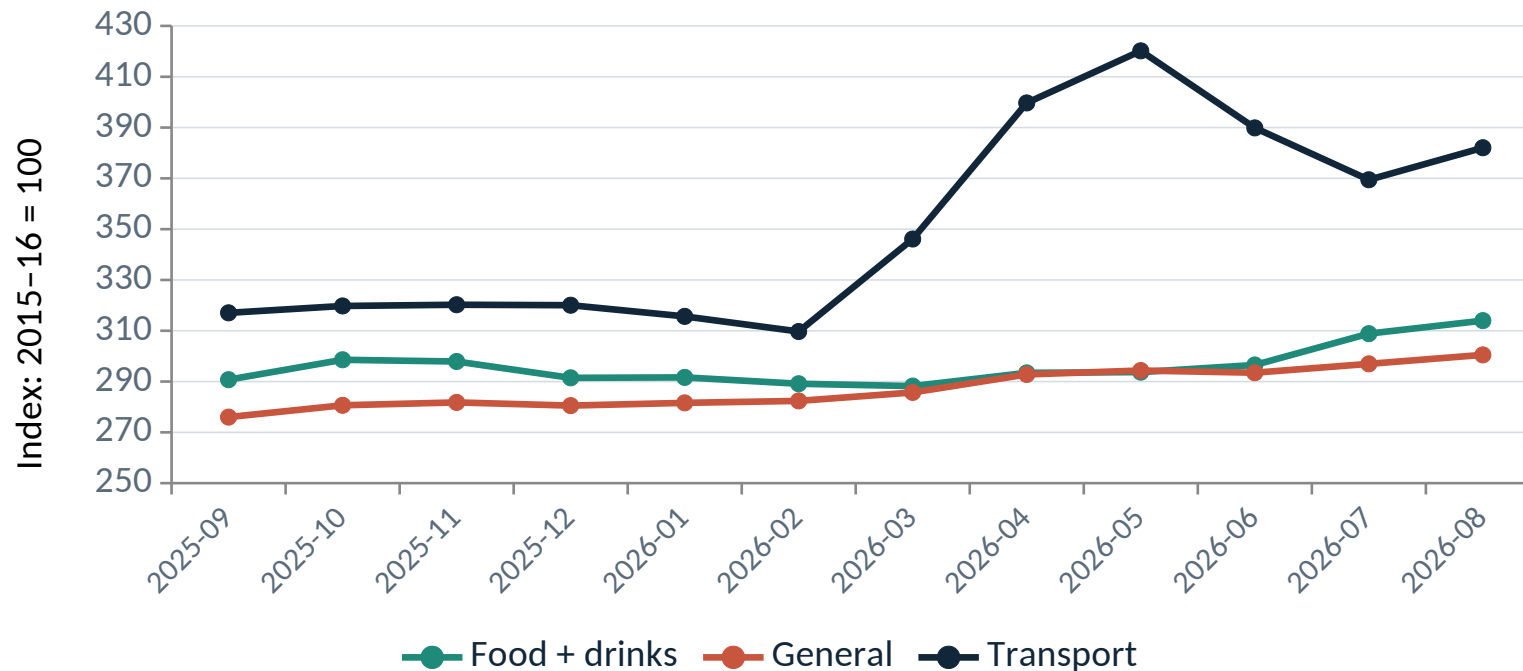
Fresh-fruit change is not apple inflation

Fresh-fruit basket	Aug 2026	Sep 2026	Change (%)
Rural retail (CPI)	251.77	245.32	-2.56
Urban retail (CPI)	253.85	253.04	-0.32
Wholesale (WPI)	297.51	272.33	-8.46

CPI / WPI fresh-fruit baskets are NOT apple-specific retail prices; monthly change $\neq$ YoY change.

Aligned months do not identify a causal effect

Three CPI baskets in matched months



Describe, do not attribute

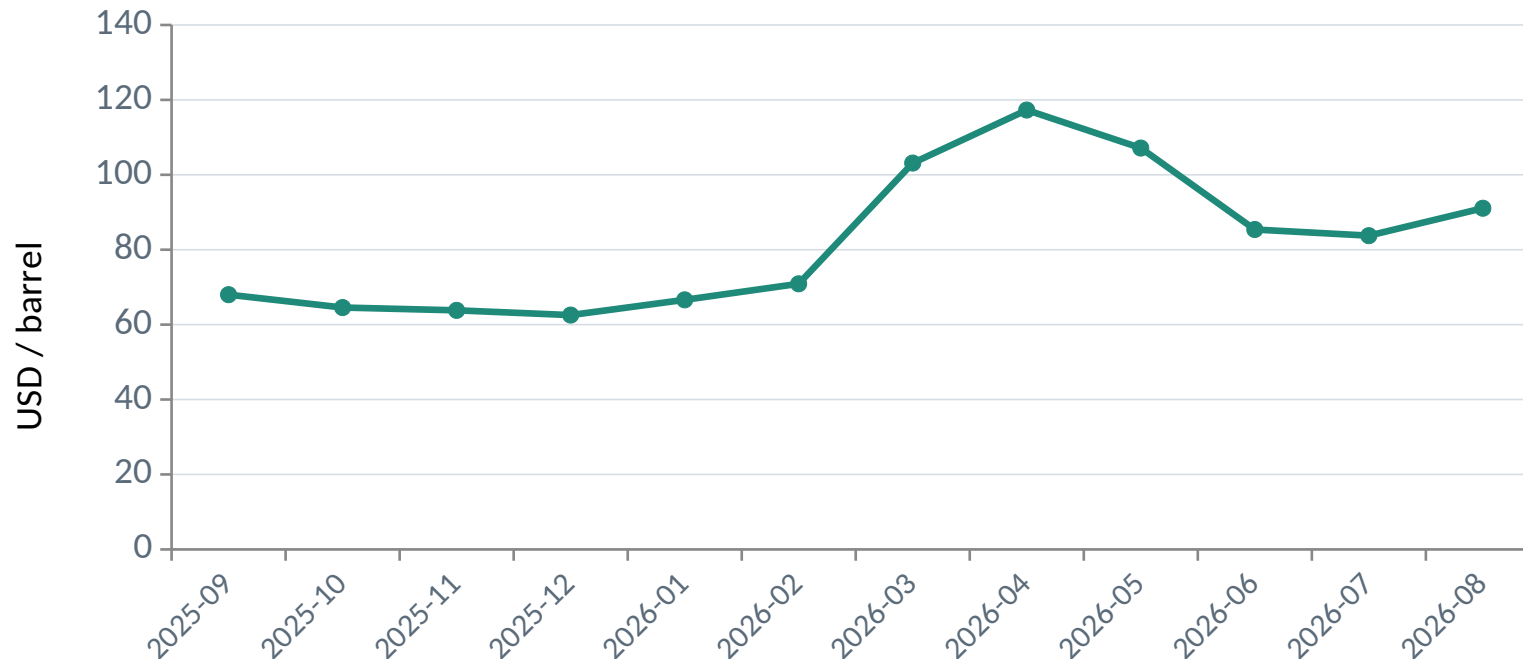
The join aligns observations.

It does not estimate oil pass-through or apple-specific costs.

Validated join · September 2025–August 2026 · shared timing is not a counterfactual.

Keep the oil benchmark on its own scale

Brent across the same matched months



A separate measurement

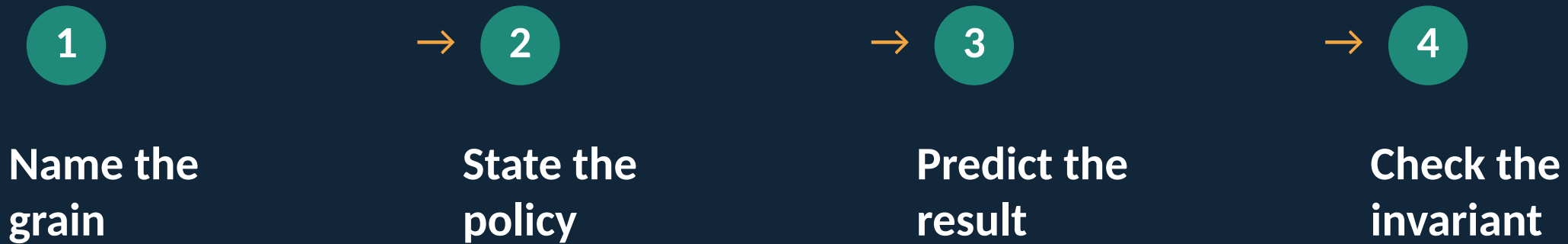
Observed-quote monthly mean.

No zero prices for weekends / closures.

No causal share is estimated.

A benchmark crude series is not a Pakistan fuel-price series.

Defend the operation, policy and pass condition



Reliable results preserve the meaning of their evidence.